PYTHON 90 DAYS

CHALLENGES

DAY-21

Python Learning Notes

Chapter 13. Modules and Packages in Python

A **module** is a Python file containing reusable code.

A **package** is a folder that contains multiple modules.

Module -> single .py file

Package -> folder containing modules

Example:

math.py -> module

my_package/ -> package

Why Modules and Packages Are Used

1. To organize large programs.
2. To reuse code.
3. To avoid writing the same logic again.
4. To separate code into meaningful files.
5. To use built-in Python features from the standard library.

13.1 Importing Modules

Importing means using code from another module.

Syntax

import module_name

1. import loads a module.
2. After importing, we can use functions, classes, and variables from that module.
3. We access module members using dot . notation.
4. Import statements are usually written at the top of the file.

Example 1

PYTHON CODE

```
import math

print(math.sqrt(25))
```

Output: 5.0 => Here, math is a module and sqrt() is a function inside it.

CodeWithNishchal

13.2 Different Ways to Import

Python provides different import styles.

Import Styles Table

Import Style	Syntax	Example	How to Use
Normal import	import module	import math	math.sqrt(25)
Import with alias	import module as alias	import math as m	m.sqrt(25)
Import specific item	from module import item	from math import sqrt	sqrt(25)
Import multiple items	from module import a, b	from math import sqrt, pow	sqrt(25)
Import all	from module import *	from math import *	sqrt(25)

Example 1

PYTHON CODE

```
from math import sqrt


print(sqrt(36))
```

Output: 6.0

Important Point

Avoid using from module import * in large programs. It can make code unclear and may cause name conflicts.

13.3 Standard Library Modules

The **standard library** is a collection of modules that come with Python. We do not need to install them separately.

Explanation

1. Standard library modules are built into Python.
2. They help with math, dates, files, operating system tasks, random values, JSON, and more.
3. We can use them by importing them.

CodeWithNishchal

Common Standard Library Modules

Module	Purpose	Example Use
math	Mathematical operations	math.sqrt(25)
random	Random values	random.randint(1, 10)
datetime	Date and time	datetime.datetime.now()
os	Operating system tasks	os.getcwd()
sys	Python runtime information	sys.version
json	Work with JSON data	json.dumps(data)
statistics	Basic statistics	statistics.mean(data)
collections	Advanced data structures	Counter, deque
itertools	Iterator tools	itertools.count()
pathlib	Work with file paths	Path("file.txt")

Example 1

```
PYTHON CODE
import random
number = random.randint(1, 10)
print(number)
```

Output: A random number between 1 and 10

Actual output changes every time because it is random.

13.4 Creating Your Own Modules

A Python file can be used as a module.

File Structure

project/
│
├── calculator.py
└── main.py

calculator.py

Code with Ishchal

```python
def add(a, b):
    return a + b
```

main.py

```python
import calculator

result = calculator.add(10, 20)
print(result)
```

Output: 30

Explanation

1. calculator.py is a module.
2. main.py imports the calculator module.
3. The function add() is accessed using calculator.add().
4. Both files should be in the same folder for this simple import to work.

13.5 Importing Specific Code from Your Own Module

Syntax

from module_name import function_name

File structure:

```
project/
│
├── calculator.py
└── main.py
```

Example 1

calculator.py

```python
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

main.py\

```python
from calculator import add
print(add(10, 20))
```

Output: 30

CodeWithNishchal

13.6 Module Search Path

When we import a module, Python searches for it in specific locations.

Explanation

Python searches in this general order:

1. Current working directory.
2. Paths listed in PYTHONPATH, if set.
3. Standard library directories.
4. Installed third-party package directories.

Syntax

```python
import sys

print(sys.path)
```

13.7 Exploring a Module with dir()

dir() shows the names available inside a module.

Syntax

dir(module_name)

Explanation

1. dir() helps us inspect a module.
2. It shows functions, variables, classes, and special names inside the module.
3. It is useful while learning or debugging.

Example 1

PYTHON CODE

```
import math

print(dir(math))
```

Output:
List of names available inside the math module
NOTE: dir() does not explain what each name does. For explanation, use help().

CodeWithNishchal

13.8 __name__ == "__main__"

This is used to control whether code should run directly or only when imported.

Syntax

```python
if __name__ == "__main__":
    statement
```

1. Every Python file has a special variable called __name__.
2. If the file is run directly, __name__ becomes "__main__".
3. If the file is imported, __name__ becomes the module name.
4. This is useful for testing module code safely.
5. It prevents some code from running during import.

Example 1

calculator.py

```python
def add(a, b):
    return a + b

if __name__ == "__main__":
    print(add(10, 20))
```

When running calculator.py directly: 30

When importing calculator.py into another file, the function is available, but the test print does not run automatically.

Flow Chart

Python file runs
  |
  v
Is file run directly?
  |
  ├── Yes -> __name__ is "__main__"
  |
  └── No -> __name__ is module name

CodeWithNishchal

13.9 Package Structure

A package is a folder containing Python modules.

Basic Package Structure

```
project/
│
├── main.py
│
└── mypackage/
    ├── __init__.py
    ├── calculator.py
    └── messages.py
```

Explanation

1. mypackage is a package.
2. calculator.py and messages.py are modules inside the package.
3. __init__.py marks the folder as a regular Python package.
4. main.py can import modules from the package.

Example 1

mypackage/calculator.py

```
PYTHON CODE
def add(a, b):
    return a + b
```

main.py

```
PYTHON CODE
from mypackage.calculator import add


print(add(10, 20))
```

Output: 30

Code With Nishchal

13.10 __init__.py

__init__.py is a special file used in Python packages.

Explanation

1. __init__.py is placed inside a package folder.
2. It tells Python that the folder is a regular package.
3. It can be empty.
4. It can also contain package-level initialization code.
5. It can control what is exposed when importing from the package.

mypackage/

├── __init__.py

├── calculator.py

└── messages.py

Example 1

mypackage/__init__.py

```
PYTHON CODE
from .calculator import add
```

main.py

```
PYTHON CODE
from mypackage import add

print(add(10, 20))
```

Output: 30

CodeWithNishchal

13.11 Absolute Imports

An absolute import uses the full path from the project/package root.

Syntax

from package.module import name

Explanation

1. Absolute imports are clear and easy to understand.
2. They show the full location of the imported module.
3. They are generally preferred in larger projects.
4. They reduce confusion compared to complex relative imports.

Example 1

Inside main.py:

from app.calculator import add

This means:

From package app,
inside module calculator,
import add

13.12 Relative Imports

A relative imports code based on the current module's location inside a package.

Syntax

from .module import name

from ..package import module

Explanation

1. Relative imports use dots.
2. One dot . means current package.
3. Two dots .. means parent package.
4. Relative imports are used inside packages.
5. They are not meant for simple standalone scripts.

CodeWithNishchal

Syntax	Meaning
.	Current package
..	Parent package
...	Grandparent package

Example Structure

```
project/
│
└── app/
    ├── __init__.py
    ├── calculator.py
    └── reports.py
```

Inside reports.py:

```
from .calculator import add
```

This means: Import add from calculator.py in the same package

Important Point

Relative imports work properly when the module is part of a package. They may fail if the file is run directly as a script.
Better way to run a package module:
python -m app.reports

13.13 Package Management Basics

Package management means installing, updating, removing, and tracking external Python packages. Python commonly uses pip for package management.

Explanation

1. The Python standard library comes with Python.
2. Third-party packages must be installed separately.
3. pip is used to install third-party packages.
4. Virtual environments are used to keep project dependencies separate.
5. requirements.txt is used to record project dependencies.

CodeWithNishchal

13.14 pip Basics

pip is Python's package installer.

Common pip Commands

Task	Command
Check pip version	python -m pip --version
Install package	python -m pip install package_name
Install specific version	python -m pip install package_name==1.2.3
Upgrade package	python -m pip install --upgrade package_name
Uninstall package	python -m pip uninstall package_name
Show installed packages	python -m pip list
Show package details	python -m pip show package_name

NOTE: Use python -m pip instead of only pip. This makes sure pip belongs to the Python version you are using.

13.15 Virtual Environment Basics

A virtual environment is an isolated Python environment for a project.

1. Different projects may need different package versions.
2. It prevents package conflicts.
3. It keeps the global Python installation clean.
4. It makes projects easier to share and manage.

Create Virtual Environment

System	Command
Windows	python -m venv venv
macOS/Linux	python3 -m venv venv

Activate Virtual Environment

CodeWithNishchal

System	Command
Windows PowerShell	venv\Scripts\Activate.ps1
Windows CMD	venv\Scripts\activate.bat
macOS/Linux	source venv/bin/activate

Deactivate

deactivate

13.16 requirements.txt

requirements.txt stores a list of packages needed for a project.

Syntax

package_name==version

Example

requests==2.32.3

numpy==2.0.0

Task	Command
Save installed packages	python -m pip freeze > requirements.txt
Install from file	python -m pip install -r requirements.txt

Explanation

1. requirements.txt helps share project dependencies.
2. Another user can install the same packages using one command.
3. It is commonly used in Python projects.

CodeWithNishchal

13.17 Third-Party Packages

Third-party packages are packages created by other developers.

They are not part of the Python standard library.

Package	Common Use
requests	Sending HTTP requests
numpy	Numerical computing
pandas	Data analysis
flask	Web development
django	Web applications
pytest	Testing

13.18 Circular Import Warning

A circular import happens when two modules import each other.

Example Structure

a.py imports b.py

b.py imports a.py

1. Circular imports can cause errors or incomplete imports.
2. They usually happen when modules depend on each other too much.
3. Good project structure helps avoid circular imports.
4. Shared code can be moved into a separate module.

Example Problem

a.py -> imports b.py

b.py -> imports a.py

Python may not finish loading one module before the other needs it.

Better Structure

common.py -> shared logic

a.py imports common.py

b.py imports common.py

CodeWithNishchal